

Programming in C

Subject Code: R23CP1401

Semester 2 · 5 Credits · Theory · CWIT R23

3 Units · 5 Lessons · 15 Quiz Questions

Comprehensive Study Notes with Curated Lectures, Study Guide & Practice Questions

Lernio AI · Learning OS 2.0

Coverage Focus

C syntax, decisions, loops, functions, arrays, strings, pointers, structures and file handling with coding practice.

Curated YouTube Lecture Resources

These playlists have been curated to match your CWIT R23 syllabus. Start with the Primary playlist, then use Alternate playlists for different explanations.

Primary Lecture 1: Programming in C - MSBTE / Diploma Playlist

Channel: YouTube playlist · Language: hi-Hinglish

URL: <https://www.youtube.com/playlist?list=PL3sp5tnO0dkOSIXP3Y2TiY7MbvQxns9X4>

Closest diploma-oriented sequence for subject and exam preparation.

Alternate Lecture 1: C Programming and Data Structures

Channel: Neso Academy · Language: en

URL: <https://www.youtube.com/playlist?list=PLBlnK6fEyqRggZZgYpPMUxdY1CYkZtARR>

Clear, detailed concept explanations; excellent for logic, pointers and fundamentals.

Alternate Lecture 2: Programming in C - K Scheme

Channel: YouTube playlist · Language: hi-Hinglish

URL: https://www.youtube.com/playlist?list=PL-iXAqgUNsnbtO_IFKziLrqRavXeu5Ben

Additional syllabus-oriented coverage and question practice.

Detailed Lesson Notes

This section contains 5 lessons across 3 units with detailed explanations, code examples, and practice quizzes.

Unit 1: C Basics and Program Structure

Weightage: 15%

Structure of a C Program

Duration: 20 min · Difficulty: easy

Overview

A C program consists of preprocessor directives, global declarations, and functions. Every C program must have a `main()` function — that's where execution begins. Understanding the structure is the first step to writing C programs.

Key Concepts

- Preprocessor directives: `#include` includes the standard I/O library
- `main()` function: the entry point of every C program — execution starts here
- Statements: each statement ends with a semicolon (`;`)
- Curly braces `{ }` define blocks of code
- `return 0;` indicates successful program termination
- `printf()` prints output to the screen; `scanf()` reads input from the keyboard
- Comments: `//` single line, `/*` multi line `*/` — ignored by the compiler

Formulas

- Σ Program structure: `#include` → global vars → functions → `main()`
- Σ `printf` format: `printf("format string", variables);`
- Σ `scanf` format: `scanf("format string", &variables);` (note the `&` for address)

Common `printf/scanf` Format Specifiers

Specifier	Data Type	Example
<code>%d</code>	<code>int</code>	<code>printf("%d", 42);</code>
<code>%f</code>	<code>float</code>	<code>printf("%f", 3.14);</code>
<code>%c</code>	<code>char</code>	<code>printf("%c", 'A');</code>

Specifier	Data Type	Example
%s	string (char array)	printf("%s", name);
%lf	double	printf("%lf", 3.14159);
%%	literal %	printf("100%%");

Forgetting the & in scanf is a common error: scanf("%d", #), NOT scanf("%d", num).

Your First C Program — Hello World with Input

```
#include <stdio.h> // preprocessor directive

int main() {
    // Variable declaration
    int age;
    char name[50];

    // Input
    printf("Enter your name: ");
    scanf("%s", name); // no & needed for strings (array name is already an address)

    printf("Enter your age: ");
    scanf("%d", &age); // & needed for int

    // Output
    printf("\nHello, %s!\n", name);
    printf("You are %d years old.\n", age);

    return 0; // success
}
```

This program asks for the user's name and age, then prints a greeting. Note: scanf needs & for int but not for string (array name is already a pointer). The \n creates a new line.

Common Mistakes

- ✗ Forgetting the & in scanf for non-array variables: scanf("%d", #) not scanf("%d", num)
- ✗ Missing semicolons at the end of statements
- ✗ Using %d for float or %f for int — format specifier must match the data type
- ✗ Forgetting #include when using printf/scanf

Exam Tips

- Every C program MUST have a main() function
- Remember the & in scanf for int, float, char — but NOT for string arrays
- Format specifiers: %d=int, %f=float, %c=char, %s=string
- Always return 0 at the end of main() to indicate success

Practice Questions

Q1. Which function is the entry point of a C program?

- A. start()
- B. begin()
- C. main()
- D. init()

Answer: C. `main()` is the entry point. Execution begins at the first statement inside `main()`. Every C program must have exactly one `main()` function.

Q2. What is wrong with: `scanf("%d", num);` where `num` is an `int`?

- A. Nothing — it's correct
- B. Missing `&` before `num`
- C. Should use `%f` instead of `%d`
- D. Missing semicolon

Answer: B. `scanf` needs the ADDRESS of the variable, not its value. For `int` variables, you must use `#`: `scanf("%d", #)`. The `&` is the 'address-of' operator. Arrays (strings) are exceptions because the array name is already an address.

Q3. Which format specifier is used to print a float value?

- A. `%d`
- B. `%f`
- C. `%c`
- D. `%s`

Answer: B. `%f` is used for float and double. `%d` is for `int`, `%c` is for `char`, `%s` is for string (`char` array).

Data Types, Variables, and Operators

Duration: 30 min · Difficulty: easy

Overview

C has several built-in data types for storing different kinds of data: `int` for integers, `float` for decimals, `char` for characters, and more. Operators perform operations on variables and values — arithmetic, relational, logical, and assignment operators.

Key Concepts

- `int`: whole numbers (4 bytes, range -2^{31} to $2^{31}-1$). Modifiers: `short`, `long`, `unsigned`.
- `float`: single-precision decimal (4 bytes, ~7 digits). `double`: double-precision (8 bytes, ~15 digits).
- `char`: single character (1 byte). Stored as ASCII integer internally.
- Arithmetic operators: `+`, `-`, `*`, `/`, `%` (modulo — remainder of division)
- Relational operators: `==`, `!=`, `<`, `>`, `<=`, `>=` (return 1 for true, 0 for false)
- Logical operators: `&&` (AND), `||` (OR), `!` (NOT)

- Assignment operators: =, +=, -=, *=, /=, %=
- Increment/decrement: ++ (add 1), -- (subtract 1). Pre: ++x (increment then use). Post: x++ (use then increment).

Formulas

Σ Size: int=4 bytes, float=4 bytes, double=8 bytes, char=1 byte

Σ Integer division: $7/2 = 3$ (not 3.5!) — both operands are int, so result is int

Σ Modulo: $7 \% 2 = 1$ (remainder of $7 \div 2$)

Σ Type casting: $(\text{float})7/2 = 3.5$ — cast one operand to float before division

C Data Types

Type	Size (bytes)	Range	Format Specifier
int	4	-2,147,483,648 to 2,147,483,647	%d
unsigned int	4	0 to 4,294,967,295	%u
short	2	-32,768 to 32,767	%hd
long	4 or 8	platform-dependent	%ld
float	4	$\pm 3.4 \times 10^{38}$ (~7 digits)	%f
double	8	$\pm 1.7 \times 10^{308}$ (~15 digits)	%lf
char	1	-128 to 127 (or 0-255 unsigned)	%c

Sizes may vary by platform. Use `sizeof(type)` to check on your system.

Operator Precedence (high to low)

Operator	Description	Associativity
() [] -> .	Function call, array, member	Left to right
! ++ -- - (type)	Unary operators	Right to left
* / %	Multiplication, division, modulo	Left to right
+ -	Addition, subtraction	Left to right
< <= > >= == !=	Relational	Left to right
&&	Logical AND	Left to right
	Logical OR	Left to right
= += -= etc.	Assignment	Right to left

Higher precedence operators are evaluated first. Use parentheses to override.

Pre vs Post Increment

Pre-increment ($++x$): increment FIRST, then use

```
int x = 5;
int y = ++x; // x becomes 6, y = 6
```

Post-increment ($x++$): use FIRST, then increment

```
int x = 5;
int y = x++; // y = 5, then x becomes 6
```

x: 5 → 6	x: 5 → 6
y: 6	y: 5

Operators in Action

```
#include <stdio.h>

int main() {
    int a = 10, b = 3;

    // Arithmetic
    printf("a + b = %d\n", a + b); // 13
    printf("a - b = %d\n", a - b); // 7
    printf("a * b = %d\n", a * b); // 30
    printf("a / b = %d\n", a / b); // 3 (integer division!)
    printf("a %% b = %d\n", a % b); // 1 (remainder)

    // Type casting
    printf("(float)a / b = %f\n", (float)a / b); // 3.333333

    // Relational (return 1 or 0)
    printf("a > b: %d\n", a > b); // 1 (true)
    printf("a == b: %d\n", a == b); // 0 (false)

    // Logical
    printf("(a > 5) && (b < 5): %d\n", (a > 5) && (b < 5)); // 1 (both true)
    printf("(a < 5) || (b < 5): %d\n", (a < 5) || (b < 5)); // 1 (one is true)

    // Increment/decrement
    int x = 5;
    printf("x++ = %d\n", x++); // 5 (print then increment)
    printf("x = %d\n", x); // 6 (now incremented)
    printf("++x = %d\n", ++x); // 7 (increment then print)

    return 0;
}
```

Note that $10/3 = 3$ (integer division — the decimal part is truncated). To get a decimal result, cast one operand to float: $(float)10/3 = 3.33$. Also note the difference between $x++$ (post) and $++x$ (pre).

Common Mistakes

- ✗ Using `/` for integer division expecting a decimal result: $7/2 = 3$, not 3.5
- ✗ Confusing `=` (assignment) with `==` (comparison): if `(x = 5)` assigns 5 to `x`, it doesn't compare!
- ✗ Using `%` with float/double — modulo only works with integers
- ✗ Forgetting that relational operators return 1 (true) or 0 (false), not 'true'/'false'

Exam Tips

- Integer division: $7/2 = 3$ (not 3.5). Cast to float: $(\text{float})7/2 = 3.5$
- $=$ is assignment, $==$ is comparison. This is a very common exam question!
- Pre-increment ($++x$): change then use. Post-increment ($x++$): use then change.
- Know the operator precedence: $*$ / $\%$ before $+$ -, relational before $\&\&$, $\&\&$ before $\|$
- $\text{sizeof}(\text{int}) = 4$ bytes, $\text{sizeof}(\text{char}) = 1$ byte, $\text{sizeof}(\text{float}) = 4$ bytes

Practice Questions

Q1. What is the output of: `int a = 7, b = 2; printf("%d", a / b);`

- A.** 3
- B.** 3.5
- C.** 4
- D.** 3.0

Answer: A. Since both a and b are int, integer division is performed. $7/2 = 3$ (the decimal part is truncated, not rounded). To get 3.5, you'd need: $(\text{float})7/2$.

Q2. What is the difference between `x++` and `++x`?

- A.** No difference
- B.** `x++` increments after use, `++x` increments before use
- C.** `x++` is for float, `++x` is for int
- D.** `x++` is faster than `++x`

Answer: B. `x++` (post-increment): use the current value first, then increment. `++x` (pre-increment): increment first, then use the new value. Example: `int x=5; int y=x++` gives `y=5, x=6`. `int y=++x` gives `y=6, x=6`.

Q3. What does the expression `(5 > 3) && (2 < 1)` evaluate to?

- A.** 1 (true)
- B.** 0 (false)
- C.** Error
- D.** 5

Answer: B. $(5 > 3)$ is true (1). $(2 < 1)$ is false (0). $1 \ \&\& \ 0 = 0$ (false). The AND operator requires BOTH conditions to be true.

Unit 2: Control Structures

Weightage: 20%

Decision Making — if-else and switch

Duration: 25 min · Difficulty: easy

Overview

Control structures allow a program to make decisions and repeat actions. The if-else statement executes different code based on a condition. The switch statement selects one of many code blocks based on a value. These are the building blocks of logic in C.

Key Concepts

- if: execute code only if a condition is true
- if-else: execute one block if true, another if false
- if-else if-else: check multiple conditions in order
- switch: select one of many blocks based on a single value (int or char only)
- break in switch: exits the switch — without it, execution falls through to the next case
- default in switch: executed if no case matches
- Ternary operator: condition ? value_if_true : value_if_false (shorthand for if-else)

Formulas

Σ if: if (condition) { /* code */ }

Σ if-else: if (condition) { /* A */ } else { /* B */ }

Σ switch: switch (var) { case 1: ...; break; case 2: ...; break; default: ...; }

Σ Ternary: result = (a > b) ? a : b;

if-else vs switch

Feature	if-else	switch
Condition type	Any expression (>, <, ==, &&,)	Equality only (== with int/char)
Data types	int, float, char, etc.	int and char only (not float or string)
Range checking	Yes (if (x >= 0 && x <= 100))	No (each case is a single value)
Fall-through	No	Yes (if break is missing)

Feature	if-else	switch
Best for	Complex conditions, ranges	Many discrete values (menus, grades)

Use switch for menu-driven programs. Use if-else for complex conditions and ranges.

Grade Calculator using if-else if

```
#include <stdio.h>

int main() {
    int marks;
    printf("Enter your marks (0-100): ");
    scanf("%d", &marks);

    if (marks >= 90) {
        printf("Grade: A+\n");
    } else if (marks >= 80) {
        printf("Grade: A\n");
    } else if (marks >= 70) {
        printf("Grade: B\n");
    } else if (marks >= 60) {
        printf("Grade: C\n");
    } else if (marks >= 50) {
        printf("Grade: D\n");
    } else {
        printf("Grade: F (Fail)\n");
    }

    return 0;
}
```

The conditions are checked top to bottom. The first true condition's block is executed, and the rest are skipped. The final 'else' is the fallback when no condition matches.

Simple Calculator using switch

```
#include <stdio.h>

int main() {
    char operator;
    float a, b;

    printf("Enter expression (e.g., 5 + 3): ");
    scanf("%f %c %f", &a, &operator, &b);

    switch (operator) {
        case '+':
            printf("%.2f + %.2f = %.2f\n", a, b, a + b);
            break;
        case '-':
            printf("%.2f - %.2f = %.2f\n", a, b, a - b);
            break;
        case '*':
            printf("%.2f * %.2f = %.2f\n", a, b, a * b);
            break;
        case '/':
            if (b != 0)
                printf("%.2f / %.2f = %.2f\n", a, b, a / b);
            else
                printf("Error: Division by zero!\n");
            break;
        default:
```

```
printf("Invalid operator!\n");
}

return 0;
}
```

switch checks the operator character. Each case handles one operator. break prevents fall-through. default handles invalid operators. Note the division-by-zero check inside the '/' case.

Common Mistakes

- ✗ Using = instead of == in conditions: if (x = 5) assigns 5 to x (always true!)
- ✗ Forgetting break in switch — causes fall-through to the next case
- ✗ Not using curly braces for multi-statement if blocks
- ✗ Using switch with float or string — switch only works with int and char

Exam Tips

- ALWAYS use == for comparison, not =. if (x == 5) compares, if (x = 5) assigns.
- Don't forget break in switch cases! Without it, execution falls through.
- switch can only test equality with int and char — not float, not ranges
- The ternary operator is a shorthand: grade = (marks >= 40) ? 'P' : 'F';

Practice Questions

Q1. What happens if you forget 'break' in a switch case?

- A. Compile error
- B. Execution falls through to the next case**
- C. The program crashes
- D. Nothing — break is optional

Answer: B. Without break, execution falls through to the next case's code, even if that case doesn't match. This is called 'fall-through'. Sometimes it's intentional, but usually it's a bug.

Q2. What is the value of x after: int x = 5; if (x = 10) { x++; }

- A. 5
- B. 6
- C. 11**
- D. 10

Answer: C. x = 10 is an ASSIGNMENT, not a comparison! It assigns 10 to x and evaluates to 10 (non-zero = true). So the if block executes: x becomes 11. This is a classic bug — use == for comparison.

Q3. Which data type can NOT be used in a switch statement?

- A. int
- B. char
- C. float
- D. short

Answer: C. switch only works with integer types (int, char, short, long). float and double cannot be used because switch tests exact equality, which is unreliable with floating-point numbers.

Loops — for, while, and do-while

Duration: 30 min · Difficulty: medium

Overview

Loops repeat a block of code multiple times. The 'for' loop is used when you know the number of iterations. The 'while' loop checks the condition BEFORE executing (may run 0 times). The 'do-while' loop checks AFTER executing (runs at least once). Loops are essential for processing arrays, generating patterns, and repeating calculations.

Key Concepts

- for loop: `for (init; condition; update) { }` — used when count is known
- while loop: `while (condition) { }` — checks BEFORE, may not execute at all
- do-while loop: `do { } while (condition)` — checks AFTER, runs at least once
- break: exits the loop immediately
- continue: skips the rest of the current iteration, goes to the next
- Nested loops: a loop inside another loop — used for 2D patterns, matrix operations
- Infinite loop: condition is always true — use break to exit

Formulas

Σ for: `for (int i = 0; i < n; i++) { /* runs n times */ }`

Σ while: `while (condition) { /* runs while true */ }`

Σ do-while: `do { /* runs at least once */ } while (condition);`

Σ Nested loop iterations: `outer × inner = total iterations`

Comparison of Loop Types

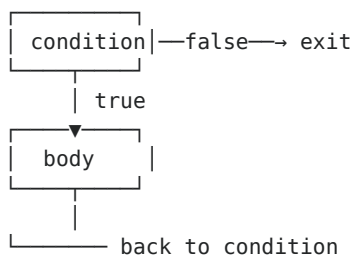
Feature	for	while	do-while
When to use	Known count	Unknown count, check first	Unknown count, run at least once

Feature	for	while	do-while
Condition check	Before each iteration	Before each iteration	After each iteration
Min executions	0	0	1
Syntax	for(init;cond;update){}	while(cond){}	do{}while(cond);
Best example	Array traversal	Reading until sentinel	Menu display

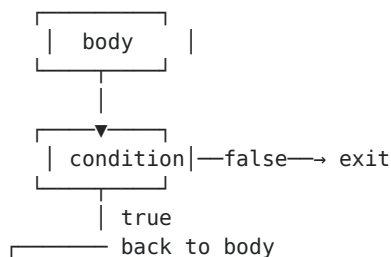
All three loops can be converted to each other, but each has its most natural use case.

Loop Flow Comparison

for/while (check first):



do-while (check last):



Key: do-while ALWAYS runs the body at least once.

Three Loops — Same Task (Print 1 to 5)

```

// Using for loop (when count is known)
for (int i = 1; i <= 5; i++) {
    printf("%d ", i); // 1 2 3 4 5
}

// Using while loop (check before)
int i = 1;
while (i <= 5) {
    printf("%d ", i); // 1 2 3 4 5
    i++;
}

// Using do-while loop (check after, runs at least once)
int j = 1;
do {
    printf("%d ", j); // 1 2 3 4 5
    j++;
} while (j <= 5);
  
```

All three produce the same output. The for loop is the most compact. The while loop separates initialization, condition, and update. The do-while guarantees at least one execution even if the condition is false initially.

Star Pattern using Nested Loops

```

#include <stdio.h>

int main() {
    int n = 5;

    // Print a right-angle triangle of stars
    for (int i = 1; i <= n; i++) { // outer: rows
        for (int j = 1; j <= i; j++) { // inner: columns
            printf("* ");
        }
        printf("\n");
    }
}
  
```

```

printf("* ");
}
printf("\n"); // new line after each row
}

return 0;
}

/* Output:
*
* *
* * *
* * * *
* * * * *
*/

```

The outer loop controls rows (1 to 5). The inner loop controls columns (1 to i). In row 1, print 1 star. In row 2, print 2 stars. And so on. The inner loop's limit depends on the outer loop variable — this is the key to triangular patterns.

Common Mistakes

- ✗ Forgetting to update the loop variable in while/do-while — causes infinite loop
- ✗ Off-by-one error: for (i = 0; i <= n; i++) runs n+1 times, not n. Use i < n.
- ✗ Forgetting the semicolon after while in do-while: do { } while (cond); ← the ; is required
- ✗ Infinite loop: while (1) { } or for (;;) { } — use break to exit

Exam Tips

- for loop: use when you know the exact number of iterations
- while: check before, may run 0 times. do-while: check after, runs at least once.
- Don't forget the semicolon after while in do-while: } while (cond);
- break exits the loop. continue skips to the next iteration.
- For nested loops, total iterations = outer_count × inner_count

Practice Questions

Q1. How many times does this loop run? for (int i = 0; i < 10; i++) { }

- A. 9
- B. 10**
- C. 11
- D. Infinite

Answer: B. i goes from 0, 1, 2, ..., 9. That's 10 values (0 through 9). The condition i < 10 is true for i=0..9, false when i=10. So the loop body executes 10 times.

Q2. What is the minimum number of times a do-while loop executes?

A. 0

B. 1

C. Depends on condition

D. Infinite

Answer: B. do-while checks the condition AFTER executing the body. So even if the condition is false initially, the body runs at least once. This is the key difference from while (which can run 0 times).

Q3. What does 'continue' do inside a loop?

A. Exits the loop completely

B. Skips the rest of the current iteration and goes to the next

C. Restarts the loop from the beginning

D. Pauses the loop

Answer: B. continue skips the remaining code in the current iteration and jumps to the next iteration. break exits the loop entirely. For example: for(i=0;i<5;i++){if(i==2)continue;printf("%d",i);} prints 0134 (skips 2).

Unit 3: Functions and Arrays

Weightage: 25%

Functions — Definition, Declaration, and Call

Duration: 30 min · Difficulty: medium

Overview

A function is a reusable block of code that performs a specific task. Functions help organize code, avoid repetition, and make programs modular. C programs are collections of functions that call each other. The `main()` function is where execution starts.

Key Concepts

- Function declaration (prototype): tells the compiler about the function's name, return type, and parameters
- Function definition: the actual implementation — contains the code that runs
- Function call: invokes the function, passing arguments
- Return type: the type of value the function returns (int, float, void, etc.)
- Parameters (formal): variables in the function definition that receive values
- Arguments (actual): values passed to the function when calling it
- Call by value: a COPY of the argument is passed — changes inside the function don't affect the original
- Call by reference: the ADDRESS of the argument is passed — changes inside the function DO affect the original
- Recursion: a function that calls itself — needs a base case to stop

Formulas

Σ Declaration: `returnType functionName(paramType param);`

Σ Definition: `returnType functionName(paramType param) { /* code */ return value; }`

Σ Call: `functionName(arguments);`

Σ Call by value: `void func(int x)` — x is a copy

Σ Call by reference: `void func(int *x)` — x is the address

Call by Value vs Call by Reference

Feature	Call by Value	Call by Reference
What is passed	Copy of the value	Address of the variable
Syntax	<code>func(x)</code>	<code>func(&x)</code> – <code>func(int *x)</code>

Feature	Call by Value	Call by Reference
Changes affect original?	No	Yes
Memory	Separate copy created	No copy – same memory
Use case	When you don't want to modify the original	When you want to modify the original or pass large data

Arrays are ALWAYS passed by reference in C (the array name decays to a pointer).

Call by Value vs. Call by Reference

```
#include <stdio.h>

// Call by value – changes don't affect original
void swapByValue(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
    // Only local copies are swapped – original unchanged
}

// Call by reference – changes DO affect original
void swapByReference(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
    // Original values are swapped via pointers
}

int main() {
    int x = 10, y = 20;

    printf("Before: x=%d, y=%d\n", x, y); // 10, 20

    swapByValue(x, y);
    printf("After swapByValue: x=%d, y=%d\n", x, y); // 10, 20 (unchanged!)

    swapByReference(&x, &y); // pass addresses
    printf("After swapByReference: x=%d, y=%d\n", x, y); // 20, 10 (swapped!)

    return 0;
}
```

swapByValue receives copies of x and y — swapping the copies doesn't affect the originals. swapByReference receives the addresses (&x, &y;) and swaps the values at those addresses — the originals ARE changed.

Recursion — Factorial

```
#include <stdio.h>

// Recursive factorial: n! = n × (n-1)!
int factorial(int n) {
    if (n <= 1) return 1; // base case: 0! = 1, 1! = 1
    return n * factorial(n - 1); // recursive call
}

/* Call trace for factorial(4):
factorial(4)
= 4 * factorial(3)
= 4 * 3 * factorial(2)
= 4 * 3 * 2 * factorial(1)
= 4 * 3 * 2 * 1
```

```

= 24
*/

int main() {
    int n = 4;
    printf("%d! = %d\n", n, factorial(n)); // 4! = 24
    return 0;
}

```

Recursion has two parts: (1) the base case ($n \leq 1 \rightarrow \text{return } 1$) that stops the recursion, and (2) the recursive case ($n * \text{factorial}(n-1)$) that reduces the problem. Without a base case, recursion never stops — stack overflow!

Common Mistakes

- ✗ Forgetting the function prototype (declaration) before `main()` if the function is defined after `main()`
- ✗ Forgetting the `&` when calling by reference: `func(&x);`, not `func(x)`
- ✗ Missing base case in recursion — causes infinite recursion and stack overflow
- ✗ Not returning a value from a non-void function — undefined behavior

Exam Tips

- Call by value: changes DON'T affect the original. Call by reference: changes DO.
- Arrays are always passed by reference in C (array name = pointer to first element)
- Recursion needs a base case — without it, infinite recursion → stack overflow
- Function prototype tells the compiler about the function before it's used
- Know how to trace recursion: draw the call stack and work backwards

Practice Questions

Q1. In call by value, what happens to the original variable?

- A. It is modified
- B. It remains unchanged**
- C. It is deleted
- D. It is copied to a new variable

Answer: B. Call by value passes a COPY of the argument. Any changes made inside the function affect only the copy, not the original variable. The original remains unchanged.

Q2. What is required to prevent infinite recursion?

- A. A loop
- B. A base case**
- C. A return statement
- D. A break statement

Answer: B. A base case is a condition that stops the recursion (e.g., if ($n \leq 1$) return 1;). Without it, the function calls itself forever, eventually causing a stack overflow.

Q3. How are arrays passed to functions in C?

- A. By value (copy)
- B. By reference (address)**
- C. By default by value
- D. Cannot be passed

Answer: B. Arrays are always passed by reference in C. When you pass an array name, it decays to a pointer to the first element. Changes made inside the function affect the original array.

Lernio AI · Learning OS 2.0 · CWIT R23

These notes are supplementary to your classroom learning. Always follow your teacher's instructions and the official CWIT syllabus.